Stateful Sequence Generator (Context Manager)

In Python, context managers are typically used for resource management, but they can also be used to manage the state of a complex iterator. Your task is to implement a resettable, stateful sequence generator within a context manager.

Task

Implement a class `sequence_generator` that maintains a running sequence of numbers. The class must act as a context manager and support multiple batches of generation within a single `with` block.

1. Constructor (`__init__`)

- Accepts `start` (int) and `step` (int).
- `start`: The first number in the sequence.
- `step`: The amount to increment the number in each step.

2. Context Management (`__enter__` and `__exit__`)

- `__enter__`: Must print `[CONTEXT] Initializing Sequence` and return the class instance.
- `__exit__`: Must print `[CONTEXT] Final Count: {total_count}` where `total_count` is the total number of elements yielded across all calls to `generate`

within that context.

3. Generation Method (`generate`)

- A generator method that takes an integer `n`.
- It should yield the next `n` numbers in the sequence.
- The state (current value) must persist between multiple calls to `generate`.

4. Reset Method (`reset`)

- Resets the sequence's current value back to the original `start`.
- Crucially, this does **not** reset the `total_count` tracked for the `__exit__` report.

Input Format

- The first line contains an integer, `start`, the beginning of the sequence.
- The second line contains an integer, `step`, the increment value.
- The third line contains an integer, `num_batches`, the number of times `generate` will be called.
- The next `num_batches` lines each contain a query. A query consists of an integer `n` (number of elements to generate) and optionally the string `RESET` if the sequence should be reset **before** generating those `n` elements.

Constraints

- $1 \leq start, step \leq 1000$
- $1 \leq num\_batches \leq 50$
- $1 \leq n \leq 100$

Output Format

- Print the initialization message from `__enter__`.
- Print each yielded number on a new line.
- Print the final summary message from `__exit__`.

Sample Input 0

```
10
5
2
3
2 RESET
```

Sample Output 0

```
[CONTEXT] Initializing Sequence
10
15
20
10
15
[CONTEXT] Final Count: 5
```

Explanation 0

1. **Initialize**: `start=10`, `step=5`. Context starts.
2. **Batch 1**: `n=3`. Yields 10, 15, 20. Current sequence value is now 25.
3. **Batch 2**: `n=2` with `RESET`. Sequence resets to 10. Yields 10, 15.
4. **Exit**: Context closes. Total elements yielded = $3 + 2 = 5$.

Sample Input 1

```
1
1
3
2
2
2
```

Sample Output 1

```
[CONTEXT] Initializing Sequence
1
2
3
4
5
6
[CONTEXT] Final Count: 6
```

Explanation 1

The sequence continues across three calls to `generate(2)` without any resets, yielding 1 through 6 in order.